

MODULE 3

Data Acquisition

TinyML — Condition Monitoring on Microcontrollers

Day 1 · Module 3 of 9

Essentials

Data quality beats model tuning. Every time.

- A mediocre model on great data outperforms a great model on garbage
- Most real-world TinyML failures are **dataset failures**:
wrong sample rate, aliasing, inconsistent mounting, label noise, leakage
- Budget your effort: **80% data, 20% model** is about right

Sampling: rate

- Sampling = measuring a continuous signal at fixed intervals **f_s**
- **Nyquist:** you can only represent frequencies below **$f_s / 2$**
- Everything above $f_s/2$ doesn't disappear — it folds back as a lie, Aliasing

Signal	Bandwidth of interest	Our f_s
Fan vibration	~20–40 Hz fundamental + harmonics	100-250 Hz
Fan acoustics	up to ~8 kHz	16 kHz

Aliasing — the signal that lies

- A 60 Hz vibration sampled at 100 Hz **shows up as 40 Hz**. Plausible. Wrong.
- You cannot detect aliasing after the fact — the data looks *fine*
- Defenses:
 - sample comfortably above $2\times$ the highest *real* frequency
 - know your sensor's bandwidth (LIS3DH internal filtering helps)
 - sanity-check: does the spectrum move as expected when you change the fan speed?

Windowing

- Models don't eat endless streams — they eat **fixed-size windows**
- Window length = how much context the model sees per decision
 - fan vibration @ 100 Hz: 1–2 s windows → 100–200 samples × 3 axes
 - audio @ 16 kHz: 1 s → 16,000 samples
- **Overlap** (e.g. 50%) = more training windows from the same recording
- Window length is a *hyperparameter you choose at data-collection time*

Range & resolution

- LIS3DH at **±4 g**, 16-bit-ish output → plenty of headroom for fan vibration
- Clipping = information destroyed. Check pandas `describe()` — are your min/max pinned at the range limits? Then widen the range.
- Same story for mic gain: too low = noise floor, too high = clipping

You verified exactly this in the Module 1 pandas exercise. It's a habit now.

Labels & metadata discipline

Create CSV naming rules / conventions :

- **Name by label:** `circle.1.csv` , `updown.2.csv` → in EI: label per recording
- Record **who/what/when**: which board, which mounting, which fan state
- Keep sessions short and single-label — easier to prune bad data later
- Plan the split **before** recording: train vs test = different sessions

Edge Impulse — end to end TinyML tooling

Studio (web): dataset management · labelling · DSP blocks · NN training · live testing · deployment as a C++ library

The screenshot displays the Edge Impulse Studio web interface in a browser. The URL bar shows `studio.edgeimpulse.com/studio/1036496/impulse/12/create-impulse`. The interface is divided into a left sidebar, a top header, and a main workspace.

Left Sidebar: Contains navigation links: Dashboard, Devices, Data acquisition, Experiments, EON Tuner, and a dropdown menu for `mfe-conv1d-53e`. Below the dropdown are links for Create impulse, Audio (MFE), Classifier, Anomaly detection, Retrain model, Live classification, Model testing, Perf. calibration, and Deployment. Two green arrows point to the `mfe-conv1d-53e` dropdown and the `Create impulse` link.

Top Header: Features the Edge Impulse logo and a description: "An impulse takes raw data, uses signal processing to extract features, and then uses a learning block to classify new data."

Main Workspace: Contains several configuration panels:

- Time series data (Red):** Includes input axes (audio), window size (1,000 ms), window increase (stride) (250 ms), frequency (15625 Hz), zero-pad data (checked), and train on data subset (100 %).
- Audio (MFE) (Light Blue):** Includes a name field (Audio (MFE)) and input axes (audio, checked).
- Classification (Purple):** Includes a name field (Classifier), input features (Audio (MFE), checked), and output features (4 (drone, helicopter, jet, noise)).
- Output features (Teal):** Includes a name field (5 (drone, helicopter, jet, noise, Anomaly score)) and a Save Impulse button.
- Anomaly Detection (K-means) (Purple):** Includes a name field (Anomaly detection).

A dashed box at the bottom center of the workspace contains a lightning bolt icon and the text "Add a processing block".

Edge Impulse — the Docs

Studio see: <https://docs.edgeimpulse.com/studio>

API Docs, see: <https://docs.edgeimpulse.com/apis>

Getting data in — our two paths

Path	Sensor	How
Data forwarder	accelerometer	board prints CSV over serial, CLI streams to Studio
Uploader	microphone	board dumps PCM → PC converts to WAV → CLI uploads



There is **no official EI firmware for the RAK4631** — and that's fine.

We build our own forwarders. *That is this module.*

The data forwarder protocol

It's just CSV over serial. Really.

```
0.12,-0.98,9.75\r\n  
0.14,-0.95,9.81\r\n
```

- CLI **auto-detects** the rate (by timing lines) and axis count
- Your only job: print values at a **steady rate**
- `edge-impulse-data-forwarder` → login → name axes `accX,accY,accZ` → done

Serial bandwidth budget

115200 baud $\approx$ 11,520 bytes/s. A CSV line `"1.23,-4.56,9.81\r\n"` $\approx$ 18 bytes.

Stream	Rate	Bytes/s	Verdict
Accel 3-axis CSV	100 Hz	$\sim$ 1.800	16% of budget
Accel 3-axis CSV	250 Hz	$\sim$ 4.500	32% of budget
Audio 16 kHz $\times$ 16-bit	raw	32.000	3 $\times$ over budget

Rule: keep the forwarder well below 115200 (or raise the baud) to leave headroom for upload.

Accel forwarder — firmware architecture

LIS3DH @ 100 Hz —► `micros()`-scheduled loop —► `Serial.print "x,y,z\r\n"`

- Simple polled delay followed by Accel read and Serial print
→ Could be implemented using Timer ISR + LIS3DH FIFO & ISR
- Units: m/s^2 (multiply g by 9.80665 — SI convention)



Lab checkpoint 1 — accelerometer → Edge Impulse (~30 min)

`exercises/module3-data-forwarding/accel-forwarder/`

- Flash the forwarder → **close the serial monitor** → run `edge-impulse-data-forwarder`
- Studio should report **100 Hz, 3 axes**; name them `accX, accY, accZ`
- Record ~5 samples each: `circle`, `updown`, `idle`
- **Done when:** you can see the difference between gestures in the raw waveform

Mic pipeline — record, dump, convert, upload

PDM 16 kHz —► RAM buffer (N s) —► base64 over serial —► pcm_to_wav.py —► edge-impulse-uploader
(device) (device) (slow, fine) (PC) (PC)

- $2\text{ s} \times 16\text{ kHz} \times 2\text{ bytes} = \mathbf{64\text{ KB}}$ — $\frac{1}{4}$ of total RAM. $5\text{ s} \approx$ the practical ceiling.
- Base64 = $\frac{4}{3}$ overhead but survives serial monitors and copy-paste
- Old course did hex + `xxd` — same idea, ours is scripted

Double buffering — the pattern that keeps giving

PDM+DMA fills buffer A —▶ event: A released
while CPU processes buffer B —▶ swap —▶ repeat

- Zero samples lost; CPU and peripheral never touch the same buffer
- nrfx PDM API: `buffer_requested` event → hand it the *other* buffer
- Same pattern later carries **live inference** (Day 3): DSP on B while A fills

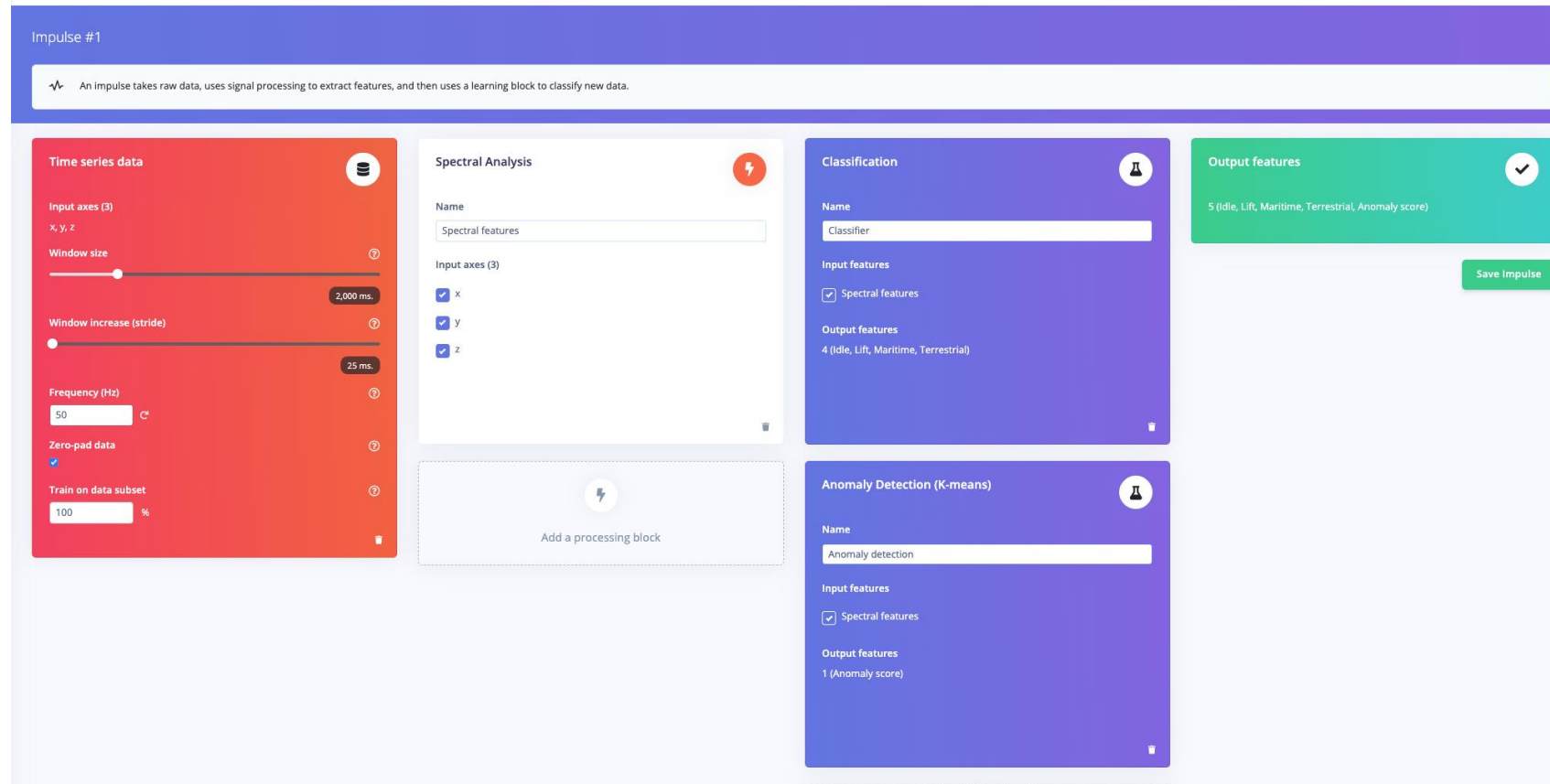


Lab checkpoint 2 — microphone → WAV → Edge Impulse (~25 min)

[exercises/module3-data-forwarding/mic-record/](#)

- Flash, capture with `pio device monitor | tee capture.txt`, send `r` to record 2 s
- `python3 pcm_to_wav.py capture.txt speech.01.wav` → **listen to it**
- `edge-impulse-uploader --category training speech.01.wav ...`
- **Done when:** you've played your clip back in Studio's browser player

Edge Impulse Studio — labelling & checking your data



- Data acquisition tab: every sample, its label, length, train/test bucket
- Click a sample → raw waveform. **Look at your data.** Every time.

What "good" first data looks like

- Forwarder: Studio shows a steady **250 Hz**, 3 axes, no gaps
- Wave the board in circles → smooth sinusoids; still → flat + gravity on one axis
- Mic: WAV plays back cleanly, speech is intelligible, no ticking (= lost buffers)
- `describe()` sanity: means near 0/0/9.8 m/s² at rest



Lab finish line — Module 3

Both checkpoints done? Check yourself out:

- [] Forwarder streams: Studio shows **250 Hz, 3 axes, no gaps**
- [] ≥ 5 labelled samples each: `circle` , `updown` , `idle`
- [] ≥ 1 audio clip recorded, converted, **listened to**, uploaded
- [] You clicked into raw samples in Studio and actually *looked*

Goal: **both sensor types, labelled, visible in your Studio project.**

Troubleshooting the pipeline

Symptom	Fix
Forwarder: "no valid sensor readings"	close the serial monitor first! One port, one owner
Detected rate $\neq$ 250 Hz	timing jitter — use the deadline scheduler, not <code>delay()</code>
Forwarder asks to log in every time	<code>edge-impulse-data-forwarder --clean</code> re-pick project
WAV sounds like a chipmunk	rate mismatch — dump was not 16 kHz, check <code>--rate</code>
Upload lands unlabeled	<code>--label</code> flag or rename file to <code>label.xx.wav</code>

Day 1 wrap-up

	Achieved
	ML fundamentals + trained NNs in Keras
✓	RAK4631 toolchain, accel, mic all alive
	Labelled sensor data flowing into Edge Impulse

Tomorrow: mount the sensor on the fan, define fault states, record a *disciplined* dataset — then make features of it.